SectraTEK

AI FOR CYBERSECURITY AND DIGITAL FORENSICS · PRACTICAL 1

Hidden in plain sight.



Steganography fundamentals, then finding hidden messages with machine learning.

Daniel Kwaku Ntiamoah Addai
Mobile Threat Research Lead, SectraTEK Lab · lab.sectratek.com

YOUR INSTRUCTOR

Daniel Kwaku Ntiamoah Addai



Mobile Threat Research Lead

SectraTEK Lab

lab.sectratek.com

IN ONE LINE

I study how phones get spied on and tampered with, and build AI tools that help examiners find the evidence.

Mobile threat research
Spyware, stalkerware and device tampering.

AI for investigators
Machine learning that speeds up the search.

Teaching
AI for Cybersecurity and Digital Forensics.

TODAY'S STORY

From a hidden message to a trained detector

1

Fundamentals

How hiding actually works

2

Before AI

How examiners looked

3

The problem

The question we must answer

4

Manual solutions

What was tried over the years

5

The gap

Where the old ways fall short

6

How AI helps

Machine learning and LLMs

7

Supervised learning

Teaching a model with labels

8

Practical

Open the notebooks in Colab

Practical 1 uses two notebooks: how_steg_works.ipynb and AI_steg_predict.ipynb.

1 FUNDAMENTALS

Some secrets are locked. Others are simply hidden.



Encryption

Scrambles the message. Everyone can see that a secret exists. Only the person with the key can read it.



Steganography

Hides the message inside something ordinary, like a photo. Nobody can even see that a secret exists.

From the Greek *steganos* (covered) and *graphia* (writing): covered writing. A locked door versus a hidden door.

1 FUNDAMENTALS

These two images are not the same



Cover image



Stego image, hides "Secret"

WHAT CHANGED

28
pixels changed, out of 262,144

1
largest change, out of 255

0
difference you can see

Both are real. The right one carries the word "Secret" in its pixels.

1 FUNDAMENTALS

To a computer, an image is a grid of numbers

223	222	224	216	208
225	230	231	214	195
202	183	144	98	74
66	43	34	25	24
25	25	25	22	20

A real 5 x 5 patch from the image used in the practical.

Pixels

Each square is one pixel. A 512 x 512 photo has 262,144 of them.

0 to 255

Each pixel is a number. 0 is black, 255 is white, everything between is a shade of gray.

8 bits

The computer stores each number as eight 1s and 0s. 208 is stored as 11010000.

1 FUNDAMENTALS

Hiding the word "Secret", in five moves

1. Spell it

Write the message: Secret

2. To bits

Turn each letter into 8 bits

3. Pick the last bit

Of each pixel, one by one

4. Swap it in

Replace that bit with a message bit

5. Read it back

Collect the last bits to recover it

This is LSB steganography: Least Significant Bit. The next slides walk through each move with the real numbers.

1 FUNDAMENTALS

Move 1 and 2: turn "Secret" into bits

S	becomes	0 1 0 1 0 0 1 1
e	becomes	0 1 1 0 0 1 0 1
c	becomes	0 1 1 0 0 0 1 1
r	becomes	0 1 1 1 0 0 1 0
e	becomes	0 1 1 0 0 1 0 1
t	becomes	0 1 1 1 0 1 0 0

WHY BITS

Every character has a number in a standard table called ASCII. S is 83, which in 8 bits is 01010011. The computer already stores text this way.

THE WHOLE MESSAGE

6 letters x 8 bits = 48 bits. We will hide these 48 bits in the last bit of the first 48 pixels, in order.

010100110110010101100011011100100110010101110100

1 FUNDAMENTALS

Move 3 and 4: swap the last bit of a pixel

ONE PIXEL, BEFORE AND AFTER

Before	1	0	0	0	0	0	0	0	= 128
After	1	0	0	0	0	0	0	**1**	= 129

the other 7 bits never change

last bit = message bit

To store a 1, set the last bit to 1. The pixel goes from 128 to 129: a change of 1 brightness level in 255.

THE KEY IDEA

The last bit is worth the least. Flipping it nudges the colour by one step, far too small for the eye.

128 10000000

129 1000000**1**

Only the last digit moved.

1 FUNDAMENTALS

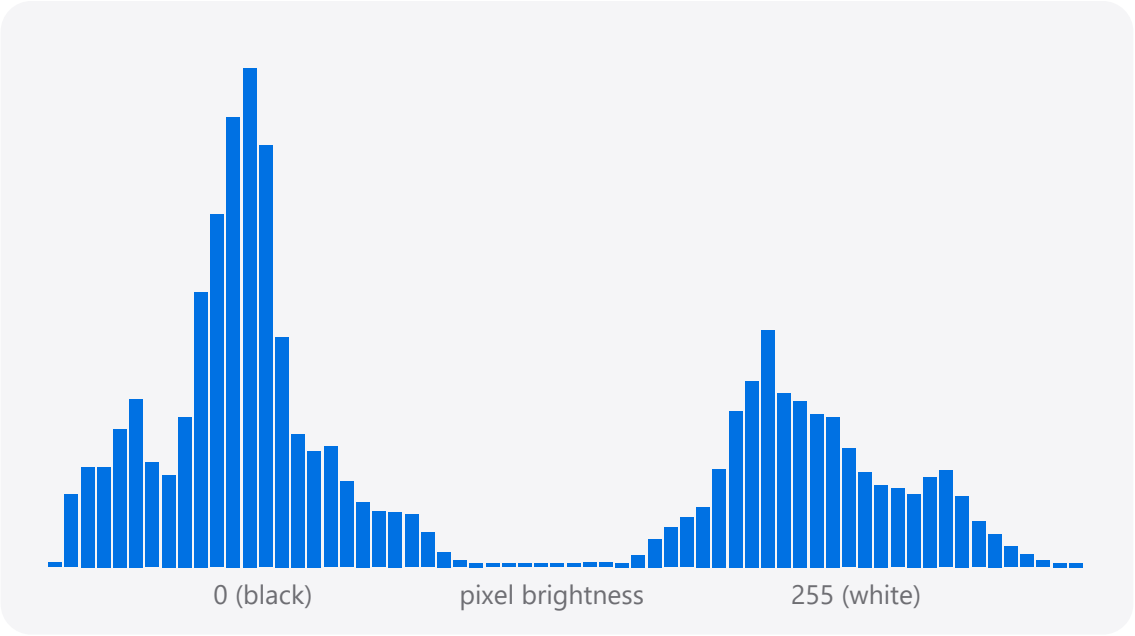
The same swap, across the first pixels

PIXEL	MESSAGE BIT	LAST BIT: BEFORE	LAST BIT: AFTER	NEW VALUE
1	0	1000000**0**	1000000**0**	128 (no change)
2	1	1000000**0**	1000000**1**	128 -> 129
3	0	0111111**0**	0111111**0**	126 (no change)
4	1	1000000**0**	1000000**1**	128 -> 129
5	0	1000000**1**	1000000**0**	129 -> 128
6	0	1000000**1**	1000000**0**	129 -> 128
7	1	1000000**1**	1000000**1**	129 (no change)
8	1	1000001**0**	1000001**1**	130 -> 131

Real values from the notebook. 'S' = 01010011 lands in the last bit of pixels 1 to 8.

1 FUNDAMENTALS

And the picture is still intact



Brightness distribution. The stego image's version is the same to the eye.

MEAN BRIGHTNESS

cover 100.4067 stego 100.4068

CONTRAST (STD DEV)

cover 72.4378 stego 72.4378

PIXELS CHANGED

cover 0 stego 28 of 262,144

THE PROBLEM IN ONE LINE

Everything an examiner would normally measure looks unchanged. That is why we need better tools.

1 FUNDAMENTALS

Getting the message back out

READ THE LAST BIT OF EACH PIXEL, IN ORDER

128	129	126	129	128	128	129	131
0	**1**	**0**	**1**	**0**	**0**	**1**	**1**

last bits: 0 1 0 1 0 0 1 1

01010011 = 83 = **S**

YOU NEED TO KNOW THE RULE

Where it starts, and how long it is. The hider and the receiver agree in advance, like a shared key.

Do this for the first 48 bits and the word "Secret" comes back, exactly. No quality is lost, because we only ever read.

1 FUNDAMENTALS

It does not have to be a word

The bits can be anything: a password, a file, or a short command an attacker wants to smuggle past a scanner. The hiding method is identical.

EXAMPLE PAYLOAD (HARMLESS DEMO)

notepad.exe

↓

11 characters = 88 bits = last bit of 88 pixels

The file still opens as a normal picture. A person viewing it sees nothing. A program written by the attacker reads the bits back and runs them.

Seen in real malware

- Payloads and commands hidden inside PNG and JPG images, then pulled down from a normal-looking website.
- Documented families have used image steganography to carry instructions and extra code past defences.
- This is why examiners treat ordinary images as something to check, not something to trust.

1 FUNDAMENTALS

Why a forensic examiner should care

Data leaving quietly

Company files tucked inside ordinary photos can slip past checks that only look for documents.

Malware and commands

Attackers hide code and instructions inside images to avoid detection, as we just saw.

Covert messages

Two people can swap harmless-looking photos that carry a hidden conversation.

The image looks normal. The file opens normally. Nothing seems wrong. That is exactly the point.

1 FUNDAMENTALS

And AI is making hiding harder to catch

Smarter placement

Instead of filling pixels in order, AI chooses the spots that disturb the statistics the least.

Learning to beat detectors

Two AIs train against each other: one hides, one seeks, until the hidden data is almost invisible.

Hiding inside AI images

A generated image can be built with the secret baked in from the start, leaving no clean original.

Automated at scale

What took an expert can now run on thousands of images, with no skill required.

For us this cuts both ways: the same machine learning that hides better also helps us detect better. That is the rest of this course.

2 BEFORE AI

How examiners looked for hidden data

Look at it

Open the image and inspect it by eye. LSB changes are invisible, so this almost never works.

Compare files

If the clean original exists, compare the two files. Usually there is no original.

Check the file

Odd file size, data after the image ends, strange metadata, or a hiding tool installed.

Run a math test

Math tests that look for the fingerprint left by one specific hiding method.

All four depend on an expert, take time per image, and each one only catches certain cases.

3 THE PROBLEM

Given one image, can we tell whether a message is hidden inside it?

It looks identical

Cover and stego images are the same to the human eye.

There is no original

We almost never have the clean version to compare against.

There are thousands

One phone can hold tens of thousands of photos to check.

Cover image = clean. **Stego image** = carries a hidden message.

4 MANUAL SOLUTIONS

What examiners built over the years

1999

Chi-square attack

Spots the even and odd pairing that simple LSB hiding leaves behind.

2001

RS analysis

Estimates how much of an image was changed by LSB hiding.

2001

Tool signatures

Scanners such as Stegdetect look for traces of known hiding programs.

2010

The BOSS contest

A public image set (BOSSbase) to test detectors fairly. Our data comes from it.

2012

Rich models

Thousands of hand-designed measurements, handed to machine learning.

THE PATTERN

Each new hiding trick needed a new hand-made test. By 2012 the field turned to machine learning to combine many measurements at once.

5 THE GAP

Where the old ways fall short

Each test was built for one trick.

Hiders changed the trick.

1. **One method, one test.** A test built for LSB replacement misses LSB matching, where pixels move up or down by 1.
2. **Small messages hide well.** The fewer bits hidden, the weaker the signal a test can find.
3. **Slow and expert-only.** Every image needs a specialist and time.
4. **It does not scale.** A single phone holds thousands of images. People cannot check them all.

6 HOW AI HELPS

Let the computer learn the pattern

Machine learning

FINDS THE HIDDEN DATA

- Learns from thousands of labelled examples
- Weighs 41 measurements at once
- Scores an image in seconds
- Gives a probability, not just yes or no

Large language models (LLMs)

HELPS THE EXAMINER

- Explain results in plain language
- Help draft notes and reports
- Help write and fix analysis code
- Do not detect hidden data themselves

AI narrows the search. The examiner still checks, explains and decides.

MACHINE LEARNING IN ONE PICTURE

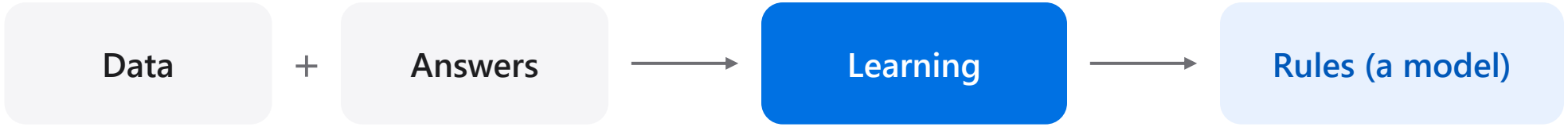
Writing the rules, or learning them

TRADITIONAL PROGRAMMING



A person writes every rule by hand.

MACHINE LEARNING



The computer works out the rules from examples.

IN OUR PRACTICAL

We give the model 20,000 images described as numbers, each labelled cover or stego. It learns the rules that separate them.

TWO WAYS A MACHINE CAN LEARN

Supervised and unsupervised learning

TODAY

Supervised learning

Learning with an answer key

Every training example comes with the right answer, called a label. The model learns to predict the label for new data.

EXAMPLES

- Spam or not spam
- Cover or stego image (Practical 1)
- Malware or clean file

Algorithms: Decision Tree, Random Forest, Logistic Regression, Support Vector Machine, k-Nearest Neighbours, Neural Network

Unsupervised learning

Learning without an answer key

No labels. The model studies the data on its own to find groups, or to spot the odd one out.

EXAMPLES

- Unusual logins at 3 am
- Grouping similar files
- Suspicious audio recordings (Practical 2)

Algorithms: k-Means, Isolation Forest, DBSCAN, and PCA for simplifying data

7 SUPERVISED LEARNING

How a supervised model learns, in five steps

1

Collect

Labelled examples: 10,000 cover and 10,000 stego images.

2

Split

70% to learn from, 30% kept hidden for the final test.

3

Train

The model studies the training set and its answers.

4

Test

Score it on the hidden 30%: images it has never seen.

5

Use

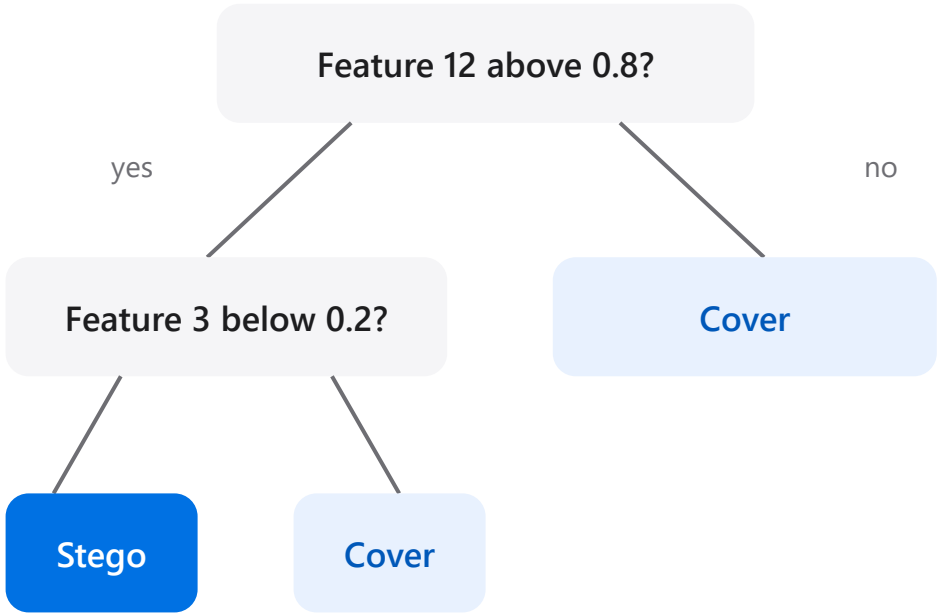
Predict new evidence, with a probability.

THINK OF IT LIKE AN EXAM

Training is revising with past papers and their answers. Testing is sitting a new paper you have never seen. Only the new paper shows what you really learned.

7 SUPERVISED LEARNING

Our classifier: a Random Forest



One decision tree: a chain of yes or no questions about the numbers.

One tree asks questions

Each question splits the images into two groups, until it reaches an answer.

A forest of 100 trees

Each tree learns from a random slice of the data, so they make different mistakes.

The trees vote

If 81 of 100 trees say stego, the probability of stego is 0.81.

7 SUPERVISED LEARNING

Turning an image into numbers the model can read

1

bit-plane correlation

How two bit layers of the pixels relate

14

neighbour patterns

How a bit layer repeats when shifted in 14 directions

5

histogram features

Even versus odd pixel values, and neighbouring values

21

texture features

Fine noise left after smoothing the image (wavelets)

41 features → Scale → PCA → 10 features → Random Forest

Scale puts every feature on the same footing. **PCA** squeezes 41 numbers into the 10 that carry most of the information. New images must go through exactly the same steps, or the model's answer is meaningless.

7 SUPERVISED LEARNING

Judge the model on images it has never seen

WHAT THE MODEL SAID

	Cover	Stego
Really cover	1,665 correct	1,239 false alarm
Really stego	512 missed	2,392 correct

Test set: 5,808 images the model never saw while learning.

70% — **test accuracy**

The number that matters. A coin flip scores 50%.

78% — **training accuracy**

Higher, because the model has seen these images. The gap is called overfitting.

About 3 mistakes in 10: a lead, never proof.

7 SUPERVISED LEARNING

The whole pipeline, mapped to the notebook



Every box is a cell you will run. Each cell in the notebook starts with its Purpose, Input and Output, in this order.

8 PRACTICAL

Let's open the notebooks.

NOTEBOOK 1

how_steg_works.ipynb

Hide a message in an image, recover it, and see why the eye cannot spot it.

Open in Colab

NOTEBOOK 2

AI_steg_predict.ipynb

Train the detector, measure it honestly, then test real images.

Open in Colab

1 **Click Open in Colab**

Nothing to install on your computer.

2 **Runtime, then Run all**

Or press Shift + Enter, one cell at a time.

3 **Read before you run**

Every step starts with Purpose, Input and Output.

Run Notebook 1 first. The first cell installs anything missing inside Colab and downloads the data for you.

8 PRACTICAL

What each step does

Notebook 1 · how_steg_works

1	Load the image and its size
2	See the pixel numbers and histogram
3	See the 8-bit binary of each pixel
4	Turn "Secret" into 48 bits
5	Hide the bits in the last bit of pixels
6-7	See the change, then recover the message
8-9	Compare histograms and simple statistics
10	Compare the 41 advanced features

Notebook 2 · AI_steg_predict

A1	Load 20,000 labelled rows
A2	Clean, split 70/30, scale, PCA
A3	Train the Random Forest, read test accuracy
B1	Load a cover and a stego image
B2	Extract the 41 features
B3	Predict both, with probabilities
B4	Optional: test your own image
B5	Scoreboard on 8 image pairs

8 PRACTICAL

What you should see, and what to discuss

0.02

probability of stego for the clean sample image

0.81

probability of stego for its stego twin

70%

test accuracy on 5,808 unseen images

9 of 14

correct on the 8-pair scoreboard (pair 2 is too dark to analyse)

DISCUSS

1. Which mistake is worse for an investigator: a false alarm or a miss?
2. What would you try to raise the accuracy?
3. Why does a JPEG photo break this model?

Numbers from the executed notebooks. Your run should match because the random seed is fixed.

SectraTEK

A finding is a lead, not proof.

Machine learning narrows the search. The examiner confirms, explains and decides.



Daniel Kwaku Ntiamoah Addai

Mobile Threat Research Lead, SectraTEK Lab

lab.sectratek.com